# MARTC: Multi-Agent Retrieval with Task-State Control

Yanlai Wu, Xu Du, Wendong Xiao, and Ying Tang, Fellow, IEEE

Abstract— Large language models can generate fluent responses, but specialized assistance like tutoring requires grounding in instructional materials, visible user or task state, and explicit answer criteria. Classical retrieval-augmented generation (RAG) usually treats grounding as a fixed retrieve-then-read step in which a query is matched against a corpus and the generator reads the returned passages. This control flow is rigid when the same surface question requires different instructional materials, response structure, or evaluation criteria across situations. We propose MARTC (Multi-Agent Retrieval with Task-State Control), a multi-agent retrieval framework that treats RAG as structured prompt construction rather than query-only passage lookup. A supervisory router decides which context requirement should dominate the prompt (planning, criteria, adaptation, or instructional materials) and whether retrieval should run; LLM-backed planning, state diagnosis, criteria analysis, and conditional retrieval prepare structured context blocks; and all intermediate outputs are reassembled into one augmented prompt for a fixed generator followed by one critique/revision pass. We instantiate and evaluate MARTC in educational tutoring, a setting where instructional materials, learner-visible state, and answer criteria must be coordinated. Controlled validation on two educational benchmarks under matched backbone, corpus, reranker, and response cap shows that MARTC improves quality while reducing token use, with additional latency reductions in some settings.

## I. INTRODUCTION

Large language models (LLMs) are increasingly used as specialized assistants because they can generate explanations, diagnose errors, and provide feedback. However, useful assistance cannot rely only on parametric knowledge. Knowledge-grounded dialogue has long shown the value of external instructional materials [1]; many assistance settings require grounding in curated resources, user-visible context, task history, and evaluation criteria to avoid fluent but misaligned guidance. Educational tutoring is a concrete and demanding instance of this problem: a tutor must coordinate course materials, worked examples, rubrics, learner-visible context, and dialogue history [2], [3].

Classical retrieval-augmented generation (RAG) has therefore become the default grounding mechanism [4]. In classical RAG, the query is embedded, the top-$k$ most similar passages are retrieved, and the LLM generates a response conditioned on those passages. Thus, relevance depends heavily on retrieval quality and query representation [5].

Specialized assistance, e.g., tutoring, is more complex than factual question answering. The appropriate response may depend on user background, current goal, task history, prior confusion, and evaluation criteria. We call these visible attributes the user's personalized learning state. A response must therefore be grounded not only in topically relevant instructional materials, but also in the user's state and task-specific requirements.

Classical RAG can include explicit state text when supplied, but appending user state, dialogue, criteria, and task constraints forces one query embedding to compress topic, level of detail, intent, and response requirements. As constraints grow, search intent can blur. The issue is therefore how a response system should organize constraints before retrieval and generation.

Therefore, we introduce MARTC (Multi-Agent Retrieval with Task-State Control), a multi-agent RAG framework for LLM-based tutoring. The core idea is to move beyond query-only retrieval by organizing the tutoring context before retrieval and generation. MARTC uses a supervisory regime design to determine what the response should prioritize. Each regime then activates the relevant specialized agents to construct the context needed for that type of tutoring response. In this way, MARTC avoids forcing all tutoring needs into a single retrieval query and instead separates search intent, learner state, answer criteria, and instructional materials into structured context blocks. Retrieval is guided by a clearer purpose, while learner-state and criteria information remain available throughout generation and post-generation critique.

Contributions. (i) We propose MARTC, a multi-agent retrieval framework with routing, specialist context construction, bounded tools, conditional retrieval, generation, and critique. (ii) We formalize transparent training-free controls over task state, answer criteria, and instructional materials. (iii) We instantiate the framework for educational tutoring and validate it against classical RAG and Agentic RAG baselines under paired infrastructure.

Yanlai Wu and Ying Tang are with the Department of Electrical and Computer Engineering, Rowan University, Glassboro, NJ 08028 USA (e-mail: wuyanl37@rowan.edu; tang@rowan.edu). Ying Tang is the corresponding author.

Xu Du is with Montclair State University (e-mail: dux3@mail.montclair.edu).

Wendong Xiao is with the School of Automation and Electrical Engineering, University of Science and Technology Beijing, Beijing 100083, P. R. China (e-mail: wdxiao@ustb.edu.cn).

## II. RELATED WORK

RAG controllers. Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on returned instructional materials [4]. This ordering is brittle when the same surface query should be constrained by visible state, answer criteria, or response form before instructional materials are selected. Recent variants improve specific control points. Self-RAG learns reflection tokens for retrieval and critique, so it cannot be adopted without training or instruction tuning the backbone [6]. CRAG evaluates already-retrieved documents and optionally expands to web search, so it corrects retrieval after the initial query rather than deciding what constraints should shape that query [7]. Adaptive-RAG trains a smaller model to classify question complexity and choose among QA strategies, which controls reasoning depth but not task state, criteria, or prompt assembly [8]. TARG is training-free and efficient, but its decision is only a retrieve-or-skip gate computed from a no-context draft; it does not build structured context before retrieval [9]. RAG diagnostics separate retrieval, ranking, and generation errors, but they diagnose failures rather than provide a controller that routes, budgets, retrieves, and assembles context [10]. The remaining gap is not another retriever; it is a controller that organizes constraints before retrieval.

Agentic retrieval. Agentic RAG and multi-agent LLM systems address the flexibility missing from fixed pipelines, but their control loops are usually open-ended. ReAct interleaves reasoning with tool actions, while AutoGen, CAMEL, and MetaGPT coordinate multiple LLM roles for broad task execution [11]–[14]. MA-RAG is closer because it decomposes ambiguous and multi-hop QA into planner, step-definition, extraction, and QA agents [15]. Its strength is iterative reasoning for difficult questions; its limitation for controlled RAG is that agents remain part of an answer-solving chain rather than a bounded prompt-construction contract.

Educational agents and evaluation. Educational systems such as AgentTutor assess learners, maintain memory, and adapt teaching strategies across multi-turn instruction [3]; evaluation work measures whether a produced tutor response is pedagogically useful [2], [16]–[19]. These works motivate our validation setting because they show that visible state and answer criteria matter. However, complete tutoring policies entangle retrieval quality with memory, dialogue policy, and teaching strategy. They therefore do not isolate the question MARTC targets: whether context construction itself can be made more flexible and efficient under a fixed generator.

## III. METHODOLOGY

Specialized assistance requires more than answering a question correctly. A system often needs to infer what the user is asking, what state or misconception is visible, what level of explanation is appropriate, what criteria should be satisfied, and whether external instructional materials are needed. These requirements make the task a context-construction problem before it is a generation problem. Simply appending these constraints to a query in classical RAG can overload the query representation, blur search intent, and retrieve instructional materials that are topically relevant but not appropriate for the requested response. Although adaptive RAG methods start to address this issue, they tend to target one control problem at a time. This motivates a unified multi-agent framework that coordinates user state, task goals, answer criteria, and instructional-material needs before generation. MARTC is designed for this need: it treats RAG as coordinated context construction rather than query-only passage lookup. As shown in Fig. 1, MARTC consists of three main components: input normalization, supervisor routing, and specialist agent coordination. Algorithm 1 gives the corresponding inference procedure from normalized input to final answer.

### A. Input Normalization

MARTC first converts each request into one normalized input entry:

$$x = (q, o, c, h, r, v). \tag{1}$$

The entry contains only information visible before generation. Here, $q$ is the question or task; $o$ is any answer-option set; $c$ is inline context already provided with the request; $h$ is recent interaction history; $r$ is explicit criteria such as a rubric or required feedback points; and $v$ stores images when present. Any field may be empty. This normalization step is deliberately separated from agent outputs: it only standardizes heterogeneous inputs so the router and specialist agents receive the same interface. This prevents topic, state, criteria, and instructional-material needs from being compressed into a single long retrieval query [3], [20].

### B. Supervisor Routing

The supervisory router is responsible for mapping each input sample to a response regime and activating the corresponding specialist agents. Given an input sample, the router examines the visible question, optional context, dialogue history, learner-state information, rubric or criteria text, and task metadata. It then estimates four types of context need: planning, criteria feedback, learner-state adaptation, and instructional-material support.

These needs correspond to four regimes. Planning (PL) is selected when the response requires an ordered structure, such as step sequencing, lesson planning, or exercise design. Criteria Feedback (CF) is selected when the response must be judged against explicit requirements, such as rubrics, scoring rules, or required feedback fields. Adaptive Response (AR) is selected when the same content should be explained differently depending on visible user state, such as grade level, prior confusion,

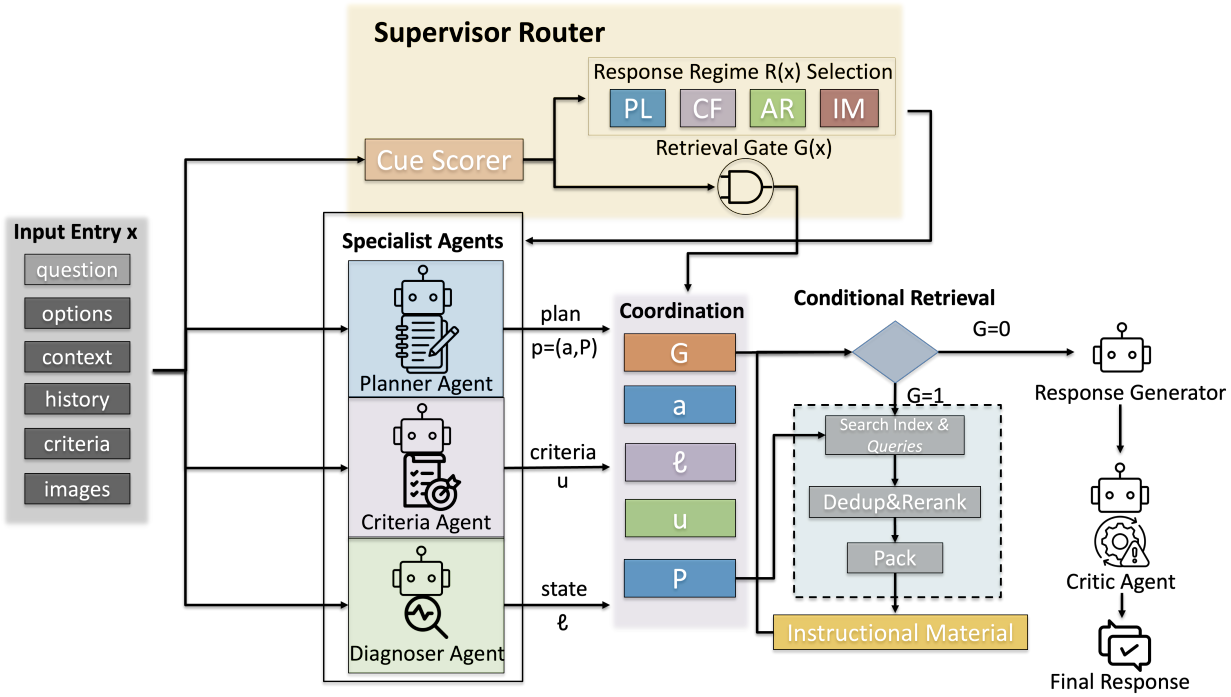


Fig. 1. Main framework of MARTC. A normalized entry $x = (q, o, c, h, r, v)$ is scored by the supervisor router, which selects the response regime $R(x) \in \{\text{PL}, \text{CF}, \text{AR}, \text{IM}\}$ and retrieval gate $G(x)$. LLM-based specialist agents then produce structured prompt blocks: planner strategy $a$ and search queries $P$, criteria summary $u$, and learner-state summary $\ell$. The coordinator merges these blocks with retrieved instructional-material snippets into instructional-material bundle $D$ when $G(x) = 1$, then sends one augmented prompt to the generator and critic.

weak points, or preferred explanation style. Instructional Materials (IM) is selected when the response should be supported by external resources, such as textbook chapters, reference documents, or worked examples.

To make this routing decision transparent and training-free, MARTC uses cue-based scores rather than a learned classifier. The router first constructs a routing string $b(x)$ by concatenating the textual fields that are visible before generation:

$$b(x) = \text{concat}(q, c, r, h), \tag{2}$$

where concat$(\cdot)$ denotes the normalization and concatenation process used to form a single routing string. Images remain available to the downstream components, but they are not converted into the textual routing score.

The router then evaluates cue families $j \in \{\text{im}, \text{cr}, \text{pl}, \text{ad}\}$, corresponding to instructional materials, criteria, planning, and adaptation. Each family has keyword set $W_j$; let $M_j(x)$ be the number of cues from $W_j$ in $b(x)$. These cue families are response-construction needs, not benchmark labels: instructional-material cues ask for source support, criteria cues ask for judgment against requirements, planning cues ask for ordered actions, and adaptation cues indicate state-aware delivery. The normalized cue score is

$$s_j(x) = \frac{M_j(x)}{|W_j|}, \tag{3}$$

where $|W_j|$ is the size of the $j$-th cue family.

The routing thresholds are introduced to prevent weak or incidental cue matches from changing the system behavior. Because the router is training-free, these thresholds serve as transparent control parameters rather than learned decision boundaries. The general threshold $\tau_{\text{mid}}$ requires repeated cues before a context need is activated. The planning threshold $\tau_{\text{plan}}$ is set more strictly because selecting the planning regime changes the response structure, not only its content.

Based on these cues and thresholds, the supervisor assigns the response regime as follows:

$$R(x) = \begin{cases} \text{PL}, & s_{\text{pl}} \geq s_{\text{cr}},\ s_{\text{pl}} \geq s_{\text{ad}},\ s_{\text{pl}} \geq \tau_{\text{plan}}, \\ \text{CF}, & s_{\text{cr}} \geq s_{\text{ad}},\ s_{\text{cr}} \geq \tau_{\text{mid}}, \\ \text{AR}, & s_{\text{ad}} \geq \tau_{\text{mid}}, \\ \text{IM}, & \text{otherwise.} \end{cases} \tag{4}$$

where $R(x)$ is the regime-selection function.

In addition to selecting the response regime, the supervisor also makes a binary retrieval decision, denoted by $G(x)$, to determine whether external instructional materials should be retrieved and added to the final prompt [6], [8], [9]:

$$G(x) = \begin{cases} 1, & s_{\text{im}} \geq \tau_{\text{mid}} \text{ or } R(x) = \text{IM}, \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

Thus, retrieval is activated if the instructional-material cue score is high enough, or if the selected regime is Instructional Materials. Otherwise, the system does not retrieve external passages.

### C. Specialist Agents and Coordination

After the supervisor assigns a regime, MARTC dispatches the normalized entry to the relevant specialist components in two stages. In the pre-generation stage, the planner agent, criteria agent, and diagnoser agent construct the response brief. Each agent independently returns a structured record, and the coordinator merges those records into one prompt.

The planner agent receives $(x, R, G)$ and returns a plan slot $p = (a, P)$, where $a$ is a short response strategy and $P$ is a short list of scoped retrieval queries. When instructional-material support is needed, $P$ specifies what external materials should be retrieved. These queries are later passed to the retriever, which is activated separately by the route or retrieval gate described in the previous subsection. The criteria agent receives $(x, R)$ and returns a criteria slot $u$, which summarizes explicit rubrics, grading rules, required output fields, or default answer-quality criteria. The diagnoser agent receives the visible question and history and returns a learner-state slot $\ell$, which summarizes level, goals, possible misconceptions, prior confusion, or preferred explanation style. MARTC also maintains an instructional-material bundle $D$ for retrieved snippets when retrieval is activated. The only user-facing output is the final answer $y$; all other slots are internal context fields used to assemble the final prompt.

In the post-generation stage, the critic agent operates after the generator produces an initial response. It is not part of the brief-construction stage. Instead, it uses the response brief, retrieved instructional materials, and draft response to perform one bounded revision pass, checking for appropriate use of instructional materials, coverage of criteria, and alignment with the learner state. If issues are detected, the critic revises the response once; otherwise, the draft is returned as the final answer.

The key point is that each regime changes how the response brief is constructed. MARTC always begins with a default task prompt containing the original input and general response requirements. When a regime is activated, MARTC adds regime-specific context blocks to the response brief.

In PL, the planner provides the main organizing structure, such as the instructional goal, step sequence, or response strategy. In CF, the criteria agent defines the evaluation standard, and the critic later checks whether the generated response satisfies it. In AR, the diagnoser supplies the learner-state summary that shapes tone, depth, scaffolding, and feedback. In IM, the planner formulates scoped search intent, and the retriever supplies instructional materials when the retrieval gate is open. The final generator prompt is assembled from the default task prompt plus the selected regime, plan, learner-state summary, criteria summary, retrieved instructional materials when available, and task instructions.

This agent-regime design avoids forcing all tutoring needs into a single retrieval query. Instead of embedding one long query containing the topic, learner state, rubric, response goal, and instructional-material need, MARTC decomposes these requirements into structured prompt blocks. Retrieval is guided by clearer search intent, while learner-state and criteria information remain available as separate context for generation and post-generation critique.

**Algorithm 1:** MARTC Inference Process

Require: entry $x = (q, o, c, h, r, v)$, index $\mathcal{I}$, thresholds $(\tau_{\text{mid}}, \tau_{\text{plan}})$.
Return: answer $y$ and internal fields $(R, G, p, \ell, u, D)$.

$b \leftarrow \text{concat}(q, c, r, h)$; $(s_{\text{im}}, s_{\text{cr}}, s_{\text{pl}}, s_{\text{ad}}) \leftarrow \text{CueScores}(b)$
$R \leftarrow R(x)$ using Eq. (4); $G \leftarrow G(x)$ using Eq. (5)
parallel do
  $p = (a, P) \leftarrow \text{Planner}(x, R, G)$
  $\ell \leftarrow \text{Diagnoser}(q, h)$
  if $r \neq \emptyset$ or $R = \text{CF}$ then $u \leftarrow \text{Criteria}(x, R)$
  else $u \leftarrow \text{DefaultCriteria}()$
  end if
end parallel
$D \leftarrow \emptyset$
if $G = 1$ then
  $D \leftarrow \text{Pack}(\text{Rerank}(\text{Dedup}(\text{Search}(\mathcal{I}, P)), q))$
end if
$y_0 \leftarrow A_{\text{gen}}(\text{Prompt}(q, o, h, p, \ell, u, D, c))$
$y \leftarrow \text{CriticOnce}(y_0, q, h, p, \ell, u, D, r)$
return $y, (R, G, p, \ell, u, D)$

## IV. VALIDATION EXPERIMENTS

We validate the framework on two public educational benchmarks used as tutoring-focused stress tests. TutorEval [2] pairs science tutoring questions with long STEM chapters and teaching key points; EduBench [17] covers diverse educational scenarios with prompts, metadata, and reference model predictions that our adapter converts into score and rubric supervision.

The primary validation compares context-assembly methods under matched infrastructure for both Qwen3.5-4B and Qwen3.5-27B-FP8, which checks whether the trends hold across backbone size. Classical RAG retrieves first from the raw standardized prompt; visible learner profile, student answer, dialogue, and criteria text remain available to its generator prompt, so the baseline is not deprived of task context. Its limitation is controller order: retrieval happens before specialist summaries can shape search. Agentic RAG uses the same specialist-coordination scaffold without MARTC's task-state retrieval control, so it tests whether agent coordination alone explains the gains. MARTC keeps the same backbone and corpus resources, then adds supervisory routing, conditional retrieval, and bounded context assembly. We report quality and cost together because only context construction changes.

### A. Implementation Details

All controllers use the same deployed Python framework, corpus, hybrid text index, reranker, response generator, critic agent, cache policy, and metric code; only context assembly differs. The planner agent, diagnoser agent, criteria agent, response generator, and critic agent use the same configured backbone; deterministic fallbacks run only when an LLM response is unavailable

or malformed. MARTC enables bounded local tool calls for specialist observations and corpus search, with no network, file mutation, or open-ended shell access. LLM endpoints are served through SGLang, whose runtime is designed for structured multi-call LLM programs and KV-cache reuse [21]. We also enable exact model-response caching for every controller because repeated prompts, fixed criteria, and repeated served requests are normal in deployed systems; a hit requires an identical full payload and is marked in the result artifacts. Therefore latency is cache-aware served-system wall time, not a cold-start model benchmark. Runs use Qwen3.5-4B and Qwen3.5-27B-FP8 [22], reasoning budget 512, generation temperature 0.15, specialist JSON temperature 0.0, and response cap 900 on benwulab-Lambda-Vector with two NVIDIA RTX 6000 Ada GPUs. Retrieval returns three packed snippets from at most four planner queries, each clipped to 6000 characters, and router thresholds are $(\tau_{\text{mid}}, \tau_{\text{plan}}) = (0.35, 0.42)$.

Evaluation is dataset-aware, and all quality scores are automatic proxies rather than official benchmark or human grades. Inspired by RAG evaluation work [10], we separate answer overlap, criteria fulfillment, instructional-material use, and resource cost. Token-F1 balances reference-token coverage against extra generated tokens; rubric coverage checks whether benchmark criteria are expressed with exact or content-bearing partial matches; latency is served wall-clock seconds per sample under the shared cache policy; and tokens count prompt plus completion tokens.

For EduBench, EB12D averages 12 normalized checks over scenario adaptation, factual/reasoning behavior, and pedagogical application, while EduAlign only measures extracted-score agreement with the benchmark consensus. For TutorEval, tutor-hit measures expert teaching-point coverage with full, partial, or zero credit.

### B. Framework Validation

Table I reports the 4B comparison, and Table II reports the 27B comparison. Both tables include Classical RAG, Agentic RAG, and MARTC. Significance markers are computed for the paired MARTC–Classical RAG comparison; Agentic RAG is included as the completed coordination baseline for interpreting whether specialist coordination alone accounts for the gains.

### C. Discussion

The results indicate that MARTC's gains come from controlling what enters the generator prompt, not from adding more context indiscriminately. TutorEval gains in tutor-hit and rubric coverage suggest that planner, criteria, and diagnoser slots preserve teaching-point and requirement coverage when instructional materials are long. EduBench gains in EB12D and rubric coverage show that task-state control is useful when requests mix scenario description, answer constraints, and desired feedback style. Agentic RAG is competitive on some quality metrics, but often spends more tokens and time because specialist outputs are not filtered through MARTC's retrieval gate and response regime.

TABLE I

4B quality and resource summary with Agentic comparison.

| Data | Metric | RAG | Agentic | MARTC | Δ |
|---|---|---|---|---|---|
| TutorEval | Token-F1 | 0.112 | 0.090 | 0.115 | $+0.004^{\star}$ |
| | Tutor hit | 0.938 | 0.932 | 0.957 | $+0.019^{\star\star}$ |
| | Rubric | 0.919 | 0.907 | 0.944 | $+0.026^{\star\star}$ |
| | Latency (s) | 51.90 | 51.05 | 49.11 | $-2.79$ |
| | Tokens | 3975 | 5959 | 3490 | $-484^{\star\star\star}$ |
| EduBench | EB12D | 0.367 | 0.398 | 0.398 | $+0.031^{\star\star\star}$ |
| | Rubric | 0.705 | 0.891 | 0.891 | $+0.186^{\star\star\star}$ |
| | EduAlign | 0.166 | 0.130 | 0.126 | $-0.040^{\star\star}$ |
| | Latency (s) | 19.83 | 23.73 | 13.90 | $-5.92^{\star\star\star}$ |
| | Tokens | 4020 | 5224 | 2226 | $-1794^{\star\star\star}$ |

Agentic is the 4B specialist-coordination baseline without MARTC's task-state retrieval control. Bold marks the best displayed value after rounding. Δ and significance compare paired 4B MARTC–RAG only: $^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Higher is better for quality; lower is better for latency and tokens.

TABLE II

27B quality and resource summary with Agentic comparison.

| Data | Metric | RAG | Agentic | MARTC | Δ |
|---|---|---|---|---|---|
| TutorEval | Token-F1 | 0.264 | 0.270 | 0.272 | $+0.008^{\star}$ |
| | Tutor hit | 0.897 | 0.902 | 0.918 | $+0.021^{\star}$ |
| | Rubric | 0.861 | 0.873 | 0.891 | $+0.030^{\star}$ |
| | Latency (s) | 41.47 | 48.31 | 40.65 | $-0.82^{\star\star}$ |
| | Tokens | 6681 | 7200 | 6730 | $+49$ |
| EduBench | EB12D | 0.355 | 0.389 | 0.425 | $+0.070^{\star\star\star}$ |
| | Rubric | 0.634 | 0.748 | 0.769 | $+0.135^{\star\star\star}$ |
| | EduAlign | 0.541 | 0.511 | 0.526 | $-0.015$ |
| | Latency (s) | 41.91 | 50.21 | 32.09 | $-9.81^{\star\star\star}$ |
| | Tokens | 6105 | 5829 | 3717 | $-2388^{\star\star\star}$ |

Agentic is the 27B specialist-coordination baseline without MARTC's task-state retrieval control. TutorEval uses $n$=400 paired samples; EduBench uses $n$=328 unique paired rows after duplicate public IDs in the 400-example slice are collapsed. Δ and significance compare paired 27B MARTC–RAG only: $^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Higher is better for quality; lower is better for latency and tokens.

The mixed EduAlign result should be interpreted narrowly: it measures agreement with extracted consensus scores, while MARTC is optimized to assemble criteria-aware responses rather than reproduce a score string. By contrast, rubric coverage and EB12D directly test whether required feedback content appears, and both improve consistently. The resource results show the trade-off: scoped planner queries and the retrieval gate usually reduce tokens and reduce 27B latency, but 27B TutorEval uses slightly more tokens because the larger model produces fuller explanations. Thus, MARTC is a controller for when and how instructional materials enter the prompt, not a universal lower-token guarantee.

## V. CONCLUSION

MARTC reframes grounding as supervised context construction rather than query-only passage lookup. With shared infrastructure, its tutoring instantiation improves fixed-generator quality and efficiency by coordinating visible task state, answer criteria, and conditional instructional materials before generation. Future work should extend beyond automatic English-only evaluation to human and multi-turn studies, stronger con-

troller baselines, learned router calibration, profile-aware reranking, privacy-preserving state inference, and deployment with appropriate data-governance and human-oversight safeguards.

## ACKNOWLEDGMENT

The authors gratefully acknowledge the support of Google Academic Research Awards.

## References

[1] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, and J. Weston, "Wizard of Wikipedia: Knowledge-powered conversational agents," in Proceedings of the International Conference on Learning Representations, 2019. [Online]. Available: https://arxiv.org/abs/1811.01241

[2] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, "Language models as science tutors," arXiv:2402.11111, 2024. [Online]. Available: https://arxiv.org/abs/2402.11111

[3] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, "AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems," arXiv:2601.04219, 2026. [Online]. Available: https://arxiv.org/abs/2601.04219

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[5] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2021. [Online]. Available: https://openreview.net/forum?id=wCu6T5xFjeJ

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: https://aclanthology.org/2024.naacl-long.389/

[9] Y. Wang, L. Wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient RAG," arXiv:2511.09803, 2026. [Online]. Available: https://arxiv.org/abs/2511.09803

[10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2408.08067

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2024. [Online]. Available: https://arxiv.org/abs/2308.08155

[13] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," 2023. [Online]. Available: https://arxiv.org/abs/2303.17760

[14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2024. [Online]. Available: https://arxiv.org/abs/2308.00352

[15] T. Nguyen, P. Chin, and Y.-W. Tai, "MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," arXiv:2505.20096, 2025. [Online]. Available: https://arxiv.org/abs/2505.20096

[16] K. K. Maurya, K. A. Srivatsa, K. Petukhova, and E. Kochmar, "Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors," in Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025. [Online]. Available: https://aclanthology.org/2025.naacl-long.57/

[17] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," arXiv:2505.16160, 2025. [Online]. Available: https://arxiv.org/abs/2505.16160

[18] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: https://aclanthology.org/2025.emnlp-main.11/

[19] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "TutorBench: A benchmark to assess tutoring capabilities of large language models," arXiv:2510.02663, 2025. [Online]. Available: https://arxiv.org/abs/2510.02663

[20] C. Borchers and T. Shou, "Can large language models match tutoring system adaptivity? A benchmarking study," in Artificial Intelligence in Education. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: https://arxiv.org/abs/2504.05570

[21] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in Advances in Neural Information Processing Systems, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html

[22] Qwen Team, "Qwen3.5: Towards native multimodal agents," Qwen Blog, Feb. 2026. [Online]. Available: https://qwen.ai/blog?id=qwen3.5